EXPERIMENT NO. 10

AIM: To generate a dataset with at least 10,000 rows, write five slow queries, analyze them using EXPLAIN, optimize them using indexing and query rewriting, and compare execution time before and after optimization.

THEORY:

Query optimization improves database performance by reducing execution time and unnecessary data access. Indexing, efficient filtering, and query rewriting help MySQL execute queries faster.
The EXPLAIN statement is used to analyze the execution plan of a query.

SQL QUERIES / PROGRAM:

Step 1: Create Database and Table

```
CREATE DATABASE OptimizationDB;

USE OptimizationDB;

CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    ProductID INT,
    Region VARCHAR(30),
    Quantity INT,
    Amount DECIMAL(10,2),
    SaleDate DATE
);
```

Step 2: Generate 10,000 Records

```
DELIMITER //

CREATE PROCEDURE GenerateSales()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 10000 DO
        INSERT INTO Sales VALUES (
```

```
            i,
            1 + (i MOD 500),
            1 + (i MOD 100),
            CASE
                WHEN i MOD 4 = 0 THEN 'North'
                WHEN i MOD 4 = 1 THEN 'South'
                WHEN i MOD 4 = 2 THEN 'East'
                ELSE 'West'
            END,
            1 + (i MOD 10),
            100 + (i MOD 5000),
            DATE_ADD('2025-01-01',
                INTERVAL (i MOD 365) DAY)
        );
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
CALL GenerateSales();
SELECT COUNT(*) AS TotalRecords FROM Sales;
```

Step 3: Write Five Slow Queries and Analyze Them

Query 1: Filter by CustomerID.

```
EXPLAIN
SELECT * FROM Sales
WHERE CustomerID = 150;
```

Query 2: Filter by Region and SaleDate.

```
EXPLAIN
```

```
SELECT * FROM Sales
WHERE Region = 'North'
AND SaleDate = '2025-03-15';
```

Query 3: Calculate total sales by region.

```
EXPLAIN
SELECT Region, SUM(Amount)
FROM Sales
GROUP BY Region;
```

Query 4: Find products with high sales amounts.

```
EXPLAIN
SELECT * FROM Sales
WHERE Amount > 4000
ORDER BY Amount DESC;
```

Query 5: Find customers whose total sales exceed a threshold.

```
EXPLAIN
SELECT CustomerID, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY CustomerID
HAVING SUM(Amount) > 10000;
```

Step 4: Optimize Queries Using Indexing

```
CREATE INDEX idx_customer
ON Sales(CustomerID);
CREATE INDEX idx_region_date
ON Sales(Region, SaleDate);
CREATE INDEX idx_amount
ON Sales(Amount);
```

```
CREATE INDEX idx_customer_amount ON
Sales(CustomerID, Amount);
```

Step 5: Rewrite Queries for Better Performance

Query 1:

```
EXPLAIN
SELECT SaleID, CustomerID, Amount
FROM Sales
WHERE CustomerID = 150;
```

Query 2:

```
EXPLAIN
SELECT SaleID, Region, SaleDate, Amount
FROM Sales
WHERE Region = 'North' AND
SaleDate = '2025-03-15';
```

Query 3:

```
EXPLAIN
SELECT Region, SUM(Amount)
FROM Sales
GROUP BY Region;
```

Query 4:

```
EXPLAIN
SELECT SaleID, ProductID, Amount
FROM Sales
WHERE Amount > 4000 ORDER
BY Amount DESC;
```

Query 5:

```
EXPLAIN
```

```
SELECT CustomerID, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY CustomerID
HAVING SUM(Amount) > 10000;
```

Step 6: Measure Execution Time

Execute the following statements before and after optimization and compare the reported execution times in MySQL Workbench or another SQL client.

```
SET PROFILING = 1;

SELECT SaleID, CustomerID, Amount
FROM Sales
WHERE CustomerID = 150;

SELECT SaleID, Region, SaleDate, Amount
FROM Sales
WHERE Region = 'North'
AND SaleDate = '2025-03-15';

SELECT Region, SUM(Amount)
FROM Sales
GROUP BY Region;

SELECT SaleID, ProductID, Amount
FROM Sales
WHERE Amount > 4000
ORDER BY Amount DESC;

SELECT CustomerID, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY CustomerID
HAVING SUM(Amount) > 10000;

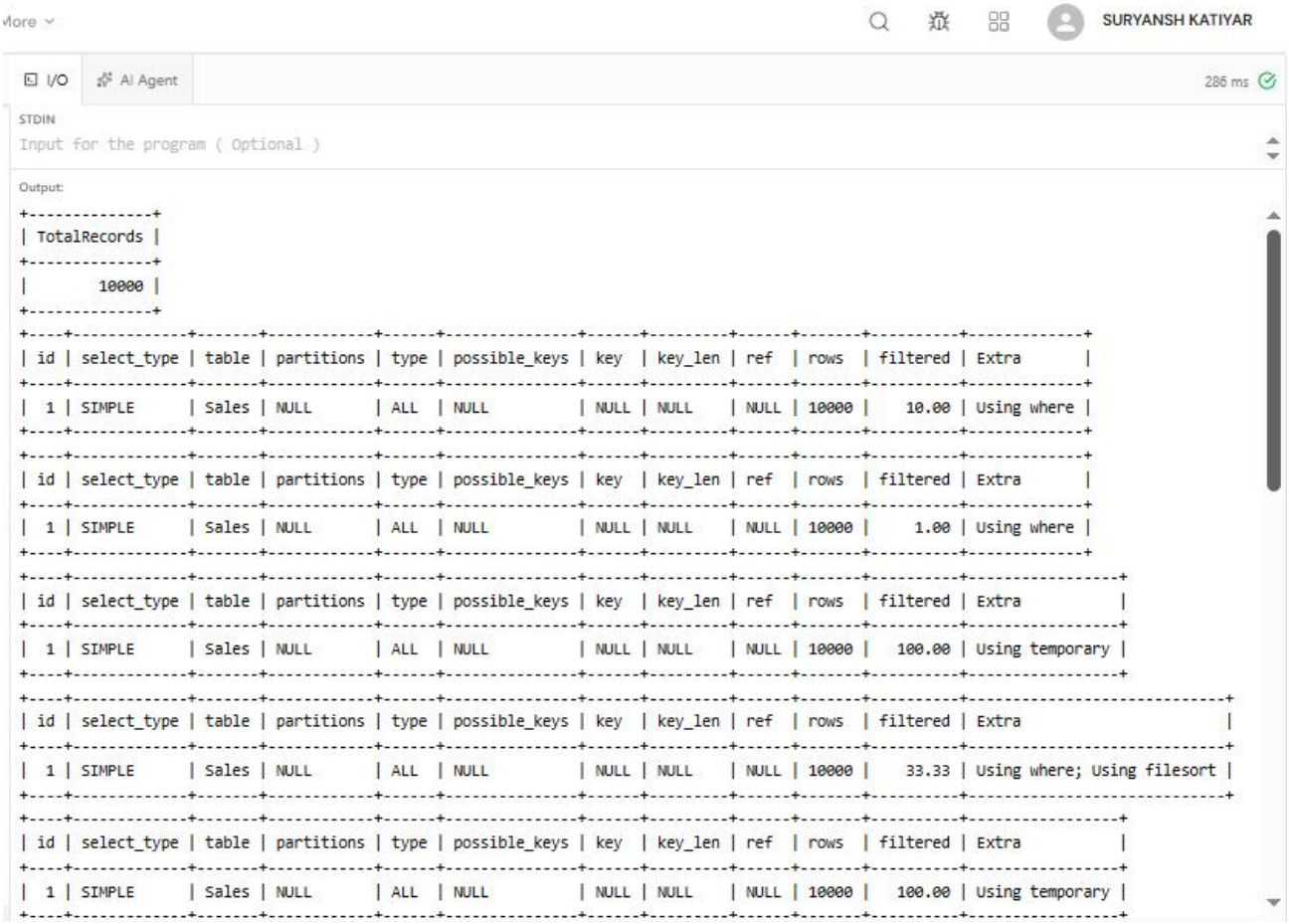
SHOW PROFILES;
```

Step 7: Verify the Dataset

SELECT COUNT(*) AS TotalRows FROM Sales;

SELECT * FROM Sales LIMIT 10;

OUTPUT:



SaleID	CustomerID	ProductID	Region	Quantity	Amount	SaleDate
1	2	2	South	2	101.00	2025-01-02
2	3	3	East	3	102.00	2025-01-03
3	4	4	West	4	103.00	2025-01-04
4	5	5	North	5	104.00	2025-01-05
5	6	6	South	6	105.00	2025-01-06
6	7	7	East	7	106.00	2025-01-07
7	8	8	West	8	107.00	2025-01-08
8	9	9	North	9	108.00	2025-01-09
9	10	10	South	10	109.00	2025-01-10
10	11	11	East	1	110.00	2025-01-11

RESULT:

Thus, a dataset containing 10,000 records was generated, and five SQL queries were analyzed and optimized using indexing and query rewriting. Execution plans and execution times were compared to evaluate the effect of optimization.